To determine whether the graph is bicolorable, we use a Breadth-First Search **(**BFS**)** traversal to attempt coloring the graph with two colors. Since the graph is connected, we can begin the BFS from node 0. We maintain a color array where each node is initially marked as uncolored (for example, using –1). We assign the first node color 0 and place it into a queue. During the BFS process, we repeatedly remove a node from the queue and examine all of its neighbors. If a neighbor is uncolored, we assign it the **opposite color** of the current node and push it into the queue to continue the traversal. However, if a neighbor already has a color and that color **matches** the color of the current node, then a coloring conflict occurs. This conflict means that two adjacent nodes require the same color, which is impossible in a valid two-coloring, and therefore the graph is not bicolorable. If the BFS finishes without finding any such conflict, then the graph can be successfully colored using two colors, which means it is bicolorable. This BFS-based method is efficient, easy to implement, and correctly identifies odd-cycle conflicts that prevent two-coloring.

BFS Steps to Check Bicoloring

Initialize all nodes as uncolored

- Imagine each node in the graph starts with no color.
- This uncolored state will help us track which nodes have already been assigned a color during BFS.

Start BFS from the first node

- Pick any node as the starting point (commonly node 0).
- Assign it the first color, say color 0.
- Add this node to a queue, which will keep track of nodes to process.

Process nodes in the BFS queue

- While the queue is not empty, take the front node from the queue and consider it the current node.
- Look at all the neighbors (adjacent nodes) of this current node.

Assign colors to neighbors
For each neighbor of the current node:

- **If the neighbor is uncolored:**
    - Assign it the opposite color of the current node.
    - Add this neighbor to the queue so its neighbors can be processed in future steps.
- **If the neighbor is already colored:**
    - Compare its color with the current node.
    - If the neighbor has the same color as the current node, a conflict occurs.
        - This conflict indicates that two adjacent nodes are forced to share the same color, which is impossible in a valid bicoloring.
        - At this point, you can conclude that the graph is not bicolorable.

- If the colors are different, there is no problem, and BFS continues.

Continue until all reachable nodes are processed

- BFS ensures that nodes are explored level by level.
- Each level of BFS naturally alternates colors, so nodes at the same level get the same color, and their neighbors get the opposite color.
- If no conflict is detected by the time the queue is empty, the graph can be successfully bicolored.

Determine the result

- **No conflicts:** All nodes were successfully colored with two colors → graph is bicolorable.
- **Conflict found:** Two adjacent nodes require the same color → graph is not bicolorable.

Input:

3

3

0 1

1 2

2 0

Stepwise BFS Coloring

Initialization

- All nodes are uncolored:
  - Node 0 → uncolored
  - Node 1 → uncolored
  - Node 2 → uncolored
- BFS queue is empty.

Step 1: Start BFS from node 0

- Assign color 0 to node 0.
- Add node 0 to the BFS queue.

Colors:

- Node 0 → 0
- Node 1 → uncolored
- Node 2 → uncolored

Queue: [0]

Step 2: Process node 0

- Neighbors of node 0: 1 and 2
- Both are uncolored → assign opposite color (1)
- Add neighbors to queue

Colors:

- Node 0 → 0
- Node 1 → 1
- Node 2 → 1

Queue: [1, 2]

Step 3: Process node 1

- Neighbors of node 1: 0 and 2
- Neighbor 0 → color 0 → fine
- Neighbor 2 → color 1 → conflict detected
    - Both node 1 and node 2 are connected and have the same color
    - Conflict means the graph cannot be bicolored

Step 4: Stop BFS

- BFS stops immediately because a conflict was found
- Graph contains an odd cycle (triangle) → impossible to 2-color

Output:

Not Bicolored.

Pseudocode:

```
WHILE true:
    READ n
    IF n == 0:
        BREAK

    READ l

    graph = array of n empty lists

    FOR i FROM 1 TO l:
        READ a, b
        ADD b to graph[a]
        ADD a to graph[b]

    color = array of size n filled with -1

    CREATE empty queue q
    color[0] = 0
    ENQUEUE q with 0

    isBipartite = true

    WHILE q is not empty AND isBipartite is true:
        u = DEQUEUE q

        FOR each neighbor v of u:
```

```
        IF color[v] == -1:
            color[v] = 1 - color[u]
            ENQUEUE q with v
        ELSE IF color[v] == color[u]:
            isBipartite = false
            BREAK

IF isBipartite:
    PRINT "BICOLORABLE."
ELSE:
    PRINT "NOT BICOLORABLE."
```